MySQL Cheat Sheet

> Compact reference for exam prep — connections, DDL/DML, joins, indexing, transactions, stored programs, backup, and optimization examples.

Connect & CLI

```
# connect (mysql client)
mysql -u username -p -h localhost -P 3306 mydb
# list databases
SHOW DATABASES;
# switch database
USE mydb;
# show tables
SHOW TABLES;
```

Data Definition (DDL)

```
-- create database
CREATE DATABASE examdb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
-- create table (InnoDB = transactional)
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  name VARCHAR(100),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- alter table
ALTER TABLE users ADD COLUMN status ENUM('active','inactive') DEFAULT 'active';
```

Tips: use `utf8mb4`, prefer `InnoDB` for transactions and foreign keys.

CRUD (DML)

```
-- insert single
INSERT INTO users (email, name) VALUES ('a@example.com', 'Asha');
-- insert multiple
```

```sql
INSERT INTO users (email, name) VALUES ('b@example.com','B'),
('c@example.com','C');
-- select
SELECT id, name FROM users WHERE status = 'active' ORDER BY created_at DESC
LIMIT 10;
-- select with projection
SELECT name, LENGTH(email) AS email_len FROM users WHERE id = 3;
-- update
UPDATE users SET name = 'A. Rao' WHERE id = 3;
-- delete
DELETE FROM users WHERE id = 10;
-- upsert (INSERT ... ON DUPLICATE KEY UPDATE)
INSERT INTO products (sku, title, stock) VALUES ('SKU1','Prod', 5)
  ON DUPLICATE KEY UPDATE stock = stock + VALUES(stock);
```

Joins & Subqueries

```sql
-- inner join
SELECT o.id, u.name, o.total
FROM orders o
JOIN users u ON u.id = o.user_id
WHERE o.status = 'shipped';

-- left join (keep all left rows)
SELECT u.id, u.name, o.total
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
WHERE o.id IS NULL; -- users without orders

-- correlated subquery
SELECT u.id, (
  SELECT COUNT(*) FROM orders o WHERE o.user_id = u.id
) AS order_count FROM users u;
```

Aggregation & GROUP BY

```sql
SELECT user_id, COUNT(*) AS orders, SUM(total) AS total_spent
FROM orders
WHERE created_at >= '2025-01-01'
GROUP BY user_id
```

```
HAVING total_spent > 1000
ORDER BY total_spent DESC;
```

Note: GROUP BY columns must be deterministic; use ONLY_FULL_GROUP_BY SQL mode for strictness.

Indexes & Explain

```
-- create index
CREATE INDEX idx_orders_user_created ON orders (user_id, created_at);
-- unique index is created by UNIQUE constraint
CREATE UNIQUE INDEX uq_products_sku ON products (sku);
-- drop index
DROP INDEX idx_orders_user_created ON orders;

-- explain
EXPLAIN SELECT * FROM orders WHERE user_id = 5 AND created_at > '2025-01-01';
```

Read EXPLAIN to check `type`, `possible_keys`, `key`, `rows` and whether index is used.

Transactions & Locking

```
-- start transaction
START TRANSACTION;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- rollback
ROLLBACK;
```

Isolation levels: `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ` (default in MySQL), `SERIALIZABLE`. Use `SELECT ... FOR UPDATE` to lock rows in InnoDB.

Stored Procedures, Functions & Triggers

```
-- stored procedure
DELIMITER $$
CREATE PROCEDURE add_user(IN p_email VARCHAR(255), IN p_name VARCHAR(100))
BEGIN
  INSERT INTO users (email, name) VALUES (p_email, p_name);
```

```
END $$
DELIMITER ;

-- function
CREATE FUNCTION fullname_len(u_id INT) RETURNS INT DETERMINISTIC
BEGIN
  DECLARE l INT;
  SELECT CHAR_LENGTH(name) INTO l FROM users WHERE id = u_id;
  RETURN l;
END;

-- trigger (after insert)
CREATE TRIGGER trg_orders_after_insert
AFTER INSERT ON orders
FOR EACH ROW
BEGIN
  UPDATE users SET last_order_at = NEW.created_at WHERE id = NEW.user_id;
END;
```

Views

```
CREATE VIEW v_user_orders AS
SELECT u.id AS user_id, u.name, COUNT(o.id) AS order_count, SUM(o.total) AS
total_spent
FROM users u
LEFT JOIN orders o ON o.user_id = u.id
GROUP BY u.id;
```

Views simplify complex queries but may have performance and updatability limits.

Backup & Restore

```
# dump entire DB
mysqldump -u root -p mydb > mydb_backup.sql
# restore
mysql -u root -p mydb < mydb_backup.sql
# dump single table
mysqldump -u root -p mydb orders > orders.sql
```

For large instances use `mysqlpump`, `xtrabackup` (Percona) or filesystem snapshots.

User Management & Privileges

```
CREATE USER 'app'@'%' IDENTIFIED BY 'S3cret!';
GRANT SELECT, INSERT, UPDATE, DELETE ON mydb.* TO 'app'@'%';
FLUSH PRIVILEGES;
```

Principle: grant least-privilege; avoid `GRANT ALL` in production.

Common Built-in Functions

- String: `CONCAT`, `SUBSTRING`, `LENGTH`, `REPLACE`, `LOWER`, `UPPER`.
- Date/time: `NOW()`, `CURDATE()`, `DATE_ADD()`, `DATEDIFF()`.
- Numeric: `ROUND`, `FLOOR`, `CEIL`, `ABS`.
- JSON: `JSON_EXTRACT`, `JSON_ARRAY`, `JSON_OBJECT`, `->` and `->>` operators.

Examples:

```
SELECT JSON_EXTRACT(metadata, '$.device.os') FROM users WHERE id = 1;
SELECT DATE_FORMAT(created_at, '%Y-%m-%d') AS d FROM orders;
```

Performance tips

- Use proper indexes for WHERE, JOIN and ORDER BY columns (match leftmost prefix for composite indexes).
- Avoid `SELECT *` in production.
- Use `EXPLAIN` and `ANALYZE` (MySQL 8+) to inspect execution plans.
- Optimize queries by reducing temp tables and filesort; add indexes or rewrite queries.
- Partition large tables by RANGE or LIST when beneficial (e.g., by date).

Security basics

- Use TLS for client connections, strong passwords, and least-privilege users.
- Disable remote root access, rotate credentials, and keep MySQL updated.

Quick Example: Users & Orders

```sql
CREATE TABLE orders (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  total DECIMAL(10,2),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_user_created (user_id, created_at),
  FOREIGN KEY (user_id) REFERENCES users(id)
) ENGINE=InnoDB;

-- get top 5 users by spending in last 30 days
SELECT u.id, u.name, SUM(o.total) AS spent
FROM users u
JOIN orders o ON o.user_id = u.id
WHERE o.created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY u.id
ORDER BY spent DESC
LIMIT 5;
```

Good luck on your exam! If you want this exported as a PDF, a printer-friendly one-page summary, or a cheat-sheet image, tell me which format and I'll prepare it.